



מסמך הארכיטקטורה - הנדסת תוכנה

מגישים:

נויה

עמית

אמיר

מבוא:

פרויקט **DiviStay** עוסק בפיתוח מערכת חכמה לניהול הזמנות לינה קבוצתיות בבתי מלון, המשלבת מנגנון מתקדם לפיצול וגביית תשלומים. מטרת המערכת היא לפתור את האתגר המוכר של ארגון חופשה משותפת, שבו מארגן הקבוצה נאלץ לרוב לקחת על עצמו את הסיכון הכלכלי, לשריין חדרים מכספו האישי, ולהתמודד באופן ידני עם איסוף הכספים משאר המשתתפים. המערכת נועדה להפוך את התהליך לשיתופי, שקוף ומאובטח מקצה לקצה.

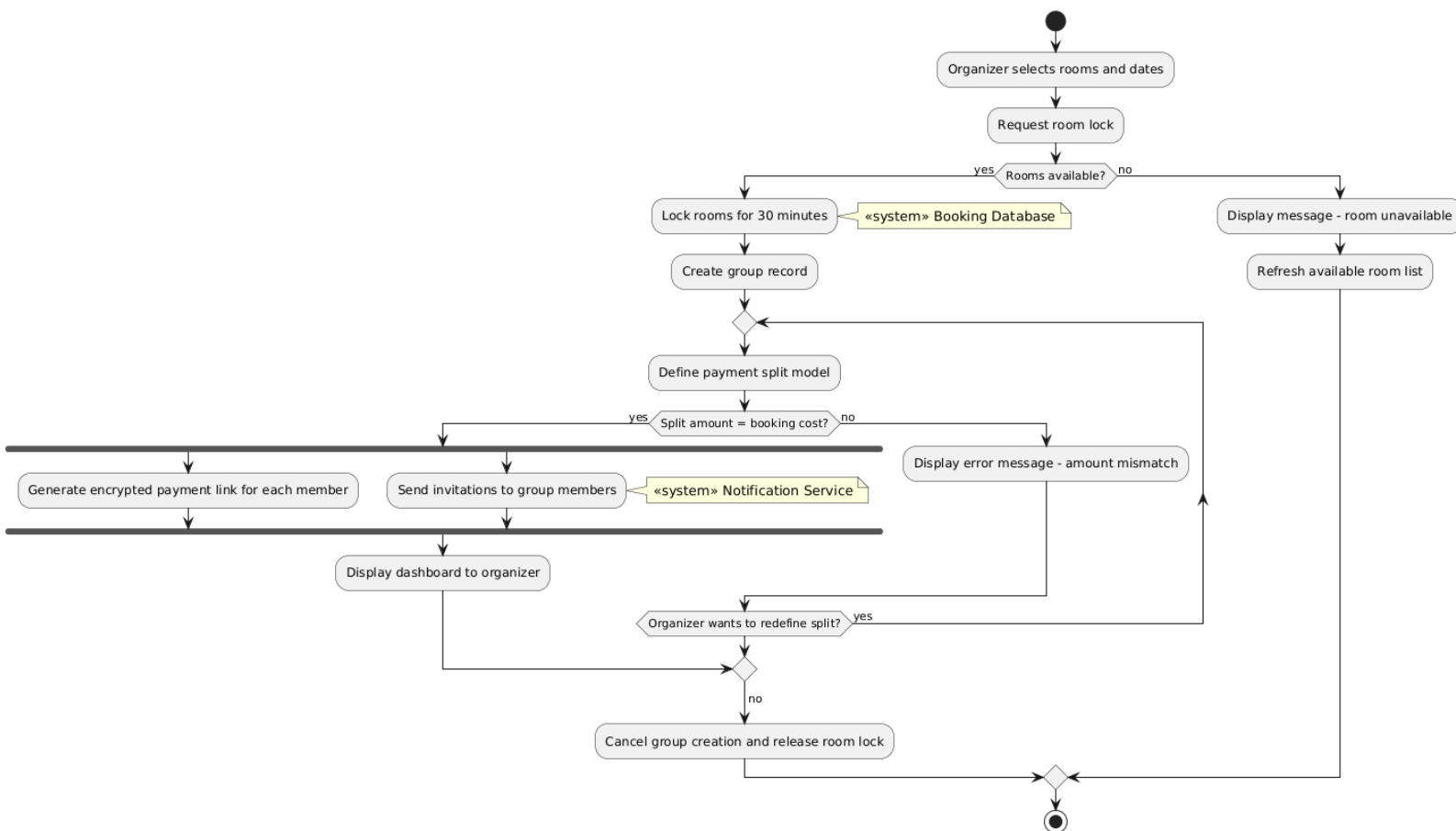
המערכת מספקת מענה אוטומטי לאורך כל ההזמנה הקבוצתית: החל מחיפוש חדרים פנויים ונעילתם הזמנית במאגר (למשך 30 דקות), דרך הגדרת מודל פיצול התשלום על ידי המארגן (חלוקה שווה, או חלוקה מותאמת אישית), ועד לשליחת קישורי תשלום מוצפנים ואישיים לכל אחד מחברי הקבוצה. המערכת מסנכרנת בזמן אמת מול שערי תשלום חיצוניים ומול מערכות ניהול המלון. הכלל המנחה הוא שרק כאשר הקבוצה כולה משלימה 100% מהתשלום הנדרש במסגרת חלון הזמן המוקצב, ההזמנה מאושרת סופית ומופקים שוברי אירוח. במידה והתשלום לא מושלם בזמן, המערכת מנהלת מנגנון ביטול אוטומטי המשחרר את החדרים חזרה למלאי, ומבטל את מסגרות האשראי של החברים שכבר שילמו.

במסמך זה, נתמקד בתכנון הארכיטקטוני וההנדסי של **DiviStay**. באמצעות סדרת מודלים ותרשימים (UML) נמחיש כיצד הדרישות הפונקציונליות מתורגמות למבנה תוכנה יציב ומודולרי. נציג את זרימת התהליכים העסקיים, את מודל הישויות, ואת חלוקת המערכת לשכבות לוגיות מופרדות היטב (MVC / Layered Architecture). בנוסף, נציג שימוש מושכל בתבניות תכנון כגון Adapter-Strategy, Factory, המבטיחות קוד גמיש, קריא, קל לתחזוקה ומוכן להרחבות עתידיות כנדרש במערכות תוכנה מודרניות.

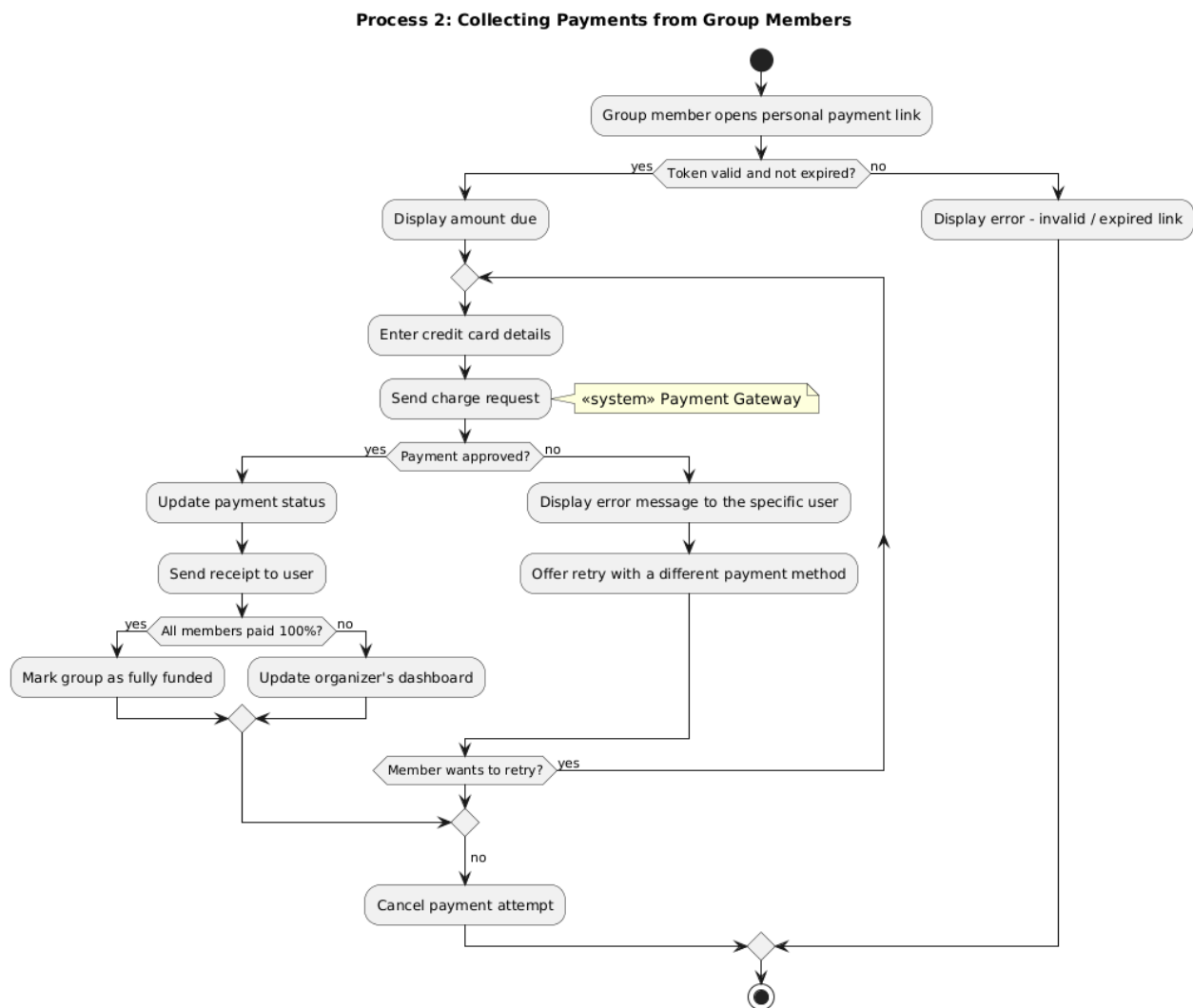
Activity Diagram

- **בחירת תאריכים וחדרים:** תרשים זה מתאר את תהליך ההזמנה הקבוצתית, המתחיל בבקשת המשתמש לנעילת תאריכים וחדרים. בשלב זה מתבצעת בדיקה וקיימים שני תרחישים:
 - ❖ **במידה שהחדרים תפוסים:** התהליך נכשל, מוצגת הודעה למשתמש ומלאי החדרים הפנויים מתרענן.
 - ❖ **במידה שהחדרים פנויים:** המערכת נועלת אותם זמנית, יוצרת רשומת קבוצה ומגדירה את מודל הפיצול. לאחר שמוודאים כי סכום החלוקה תואם במדויק לעלות ההזמנה, המערכת מפעילה במקביל יצירת קישורי תשלום מוצפנים ושליחת הזמנות לחברים. לבסוף, התהליך מסתיים בהצגת לוח הבקרה למארגן הקבוצה.

Process 1: Create Booking Group & Lock Rooms



- **חבר קבוצה פותח קישור תשלום אישי:** תרשים זה מתאר את תהליך התשלום של משתמש בודד, המתחיל כאשר הוא פותח את הקישור שנשלח אליו. בשלב הראשון מתבצע אימות לקישור וקיימים שני תרחישים:
 - ❖ **במידה והקישור אינו תקין או פג תוקף:** מוצגת הודעת שגיאה והתהליך מסתיים.
 - ❖ **במידה והקישור תקין:** מוצג הסכום לתשלום, המשתמש מזין את פרטי האשראי ונשלחת בקשת חיוב לשער התשלום. בשלב זה נבדק האם התשלום אושר:
 - **אם התשלום נכשל:** מוצגת הודעת שגיאה למשתמש ומוצעת אפשרות לניסיון חוזר עם אמצעי תשלום אחר.
 - **אם התשלום אושר:** הסטטוס מתעדכן ונשלחת קבלה. לאחר מכן, מתבצעת בדיקה האם כלל החברים שילמו את חלקם (100%). אם כן, הקבוצה כולה מסומנת כממומנת במלואה, ובמידה ולא מתבצע רק עדכון בלוח הבקרה של המארגן.



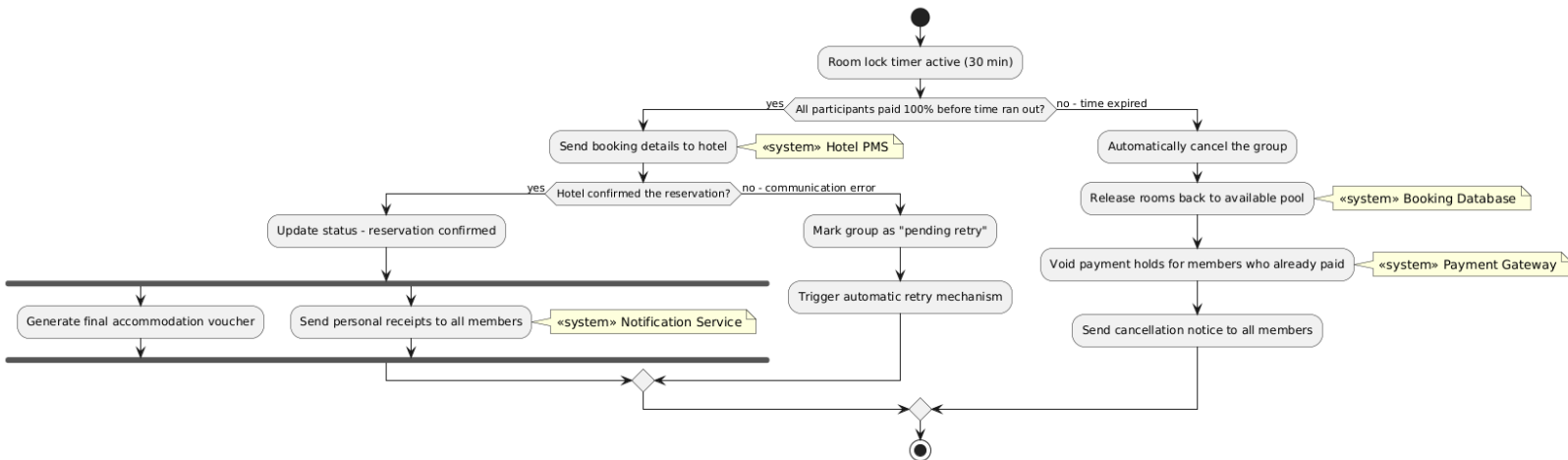
○ **טיימר נעילת החדרים פעיל (30 דקות):** תרשים זה מתאר את תהליך הבקרה שמתרחש מרגע נעילת החדרים. המערכת בודקת האם כל חברי הקבוצה השלימו את התשלום (100%) לפני תום הזמן המוקצב, ומפצלת את התהליך לשני תרחישים עיקריים:

❖ **במידה והזמן נגמר לפני השלמת התשלום:** המערכת מפעילה תהליך ביטול אוטומטי. תהליך זה כולל את שחרור החדרים חזרה למאגר הזמין, ביטול מסגרות התשלום לחברים שכבר שילמו, ושליחת הודעת ביטול לכלל חברי הקבוצה.

❖ **במידה והתשלום הושלם בזמן (100% תשלום):** המערכת שולחת את פרטי ההזמנה למלון. בשלב זה מתבצעת בדיקה נוספת לגבי אישור ההזמנה על ידי המלון:

- **אם יש שגיאת תקשורת / לא אושר:** הקבוצה מסומנת בסטטוס "ממתין לניסיון חוזר", ומופעל מנגנון ניסיון חוזר (Retry) אוטומטי.
- **אם ההזמנה אושרה:** הסטטוס מתעדכן ל"הזמנה מאושרת", והמערכת מפעילה שני תהליכים במקביל: שליחת קבלות אישיות לכל החברים והפקת שובר אירוח סופי. לאחר מכן, התהליך מסתיים בהצלחה.

Process 3: Final Reservation Confirmation or Automatic Cancellation



• Use Case Diagram

דיאגרמה זו מציגה את תרשימים ה Use Case של מערכת **DiviStay**, ומדגימה את מכלול הפעולות שכל סוג משתמש יכול לבצע במערכת:

○ שחקנים במערכת:

- ❖ **Guest**: משתמש טרם התחברות, בעל הרשאות בסיסיות לביצוע הרשמה למערכת או התחברות אליה.
- ❖ **Organizer**: משתמש מחובר בעל הרשאות ניהול, היכול לחפש חדרים, ליצור קבוצת הזמנה, להגדיר את מודל פיצול התשלום, להזמין חברים ולנהל את לוח הבקרה של הקבוצה.
- ❖ **Group Member**: משתמש מחובר הרשאי לשלם את חלקו היחסי בהזמנה ולעקוב אחר היתרה שנותרה לתשלום.

❖ **מערכת/טיימר**: שחקן לא אנושי המייצג תהליכים אוטומטיים הרצים ברקע ללא כל צורך בהתערבות משתמש.

○ גבול המערכת:

- המלבן המרכזי תוחם את גבולות מערכת ה-**DiviStay**. כלל התרחישים המופיעים בתוכו מייצגים את השירותים והיכולות שהמערכת מספקת בפועל למשתמשיה.

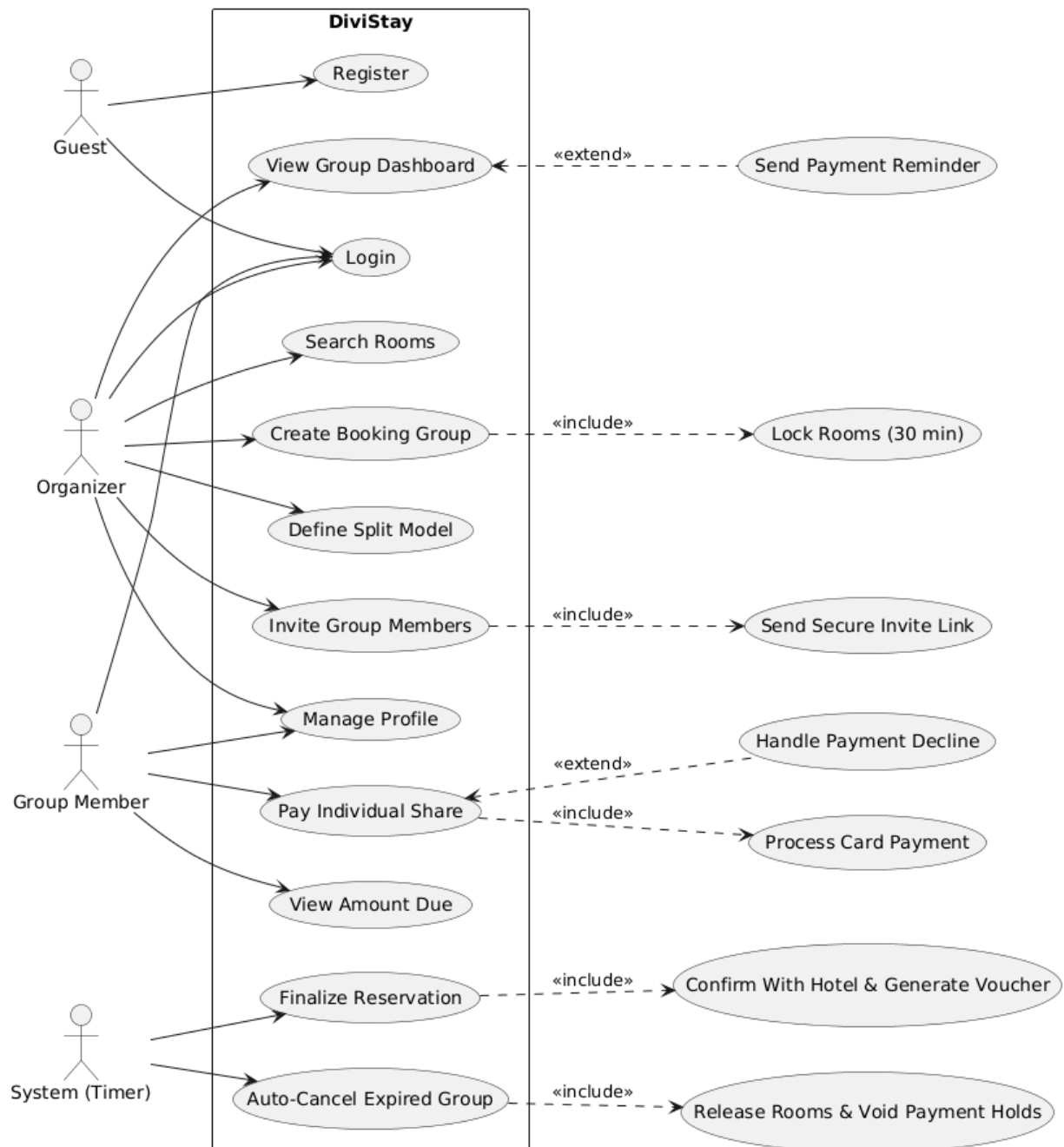
○ ניתוח סוגי הקשרים בדיאגרמה:

- ❖ **קשר ישיר**: הקווים הרגילים מחברים בין השחקנים לבין הפעולות הזמינות עבורם ישירות (למשל: יצירת קבוצה, תשלום חלק אישי, וניהול פרופיל).
- ❖ **קשר מסוג <<include>>**: מציין תלות הכרחית בה פעולה חייבת להתבצע כחלק אינטגרלי מתהליך ראשי. לדוגמה: נעילת חדרים היא שלב הכרחי המוטמע בתהליך יצירת קבוצת ההזמנה.
- ❖ **קשר מסוג <<extend>>**: מציין פעולה מותנית או אופציונלית המתרחשת רק תחת מצב מסוים, ותלויה בבחירה אנושית או בתרחיש שגיאה בתוך תהליך שהחל. לדוגמה: שליחת תזכורת תשלום (כאשר המארגן בוחר יזומה לעשות זאת מלוח הבקרה), או טיפול בסירוב תשלום (כאשר אישור האשראי נדחה באמצע עסקה).

○ תכנון ארכיטקטוני של תהליכי רקע:

תהליכים אוטומטיים כמו למשל ביטול הקבוצה בתום חלון הנעילה או אישור ההזמנה הסופי מול המלון מיוצגים כ- Use Cases עצמאיים המופעלים על ידי שחקן ה- System ולא כקשרי <<extend>> של פעולת משתמש.

Use-Case Diagram - DiviStay



• Sequence diagrams

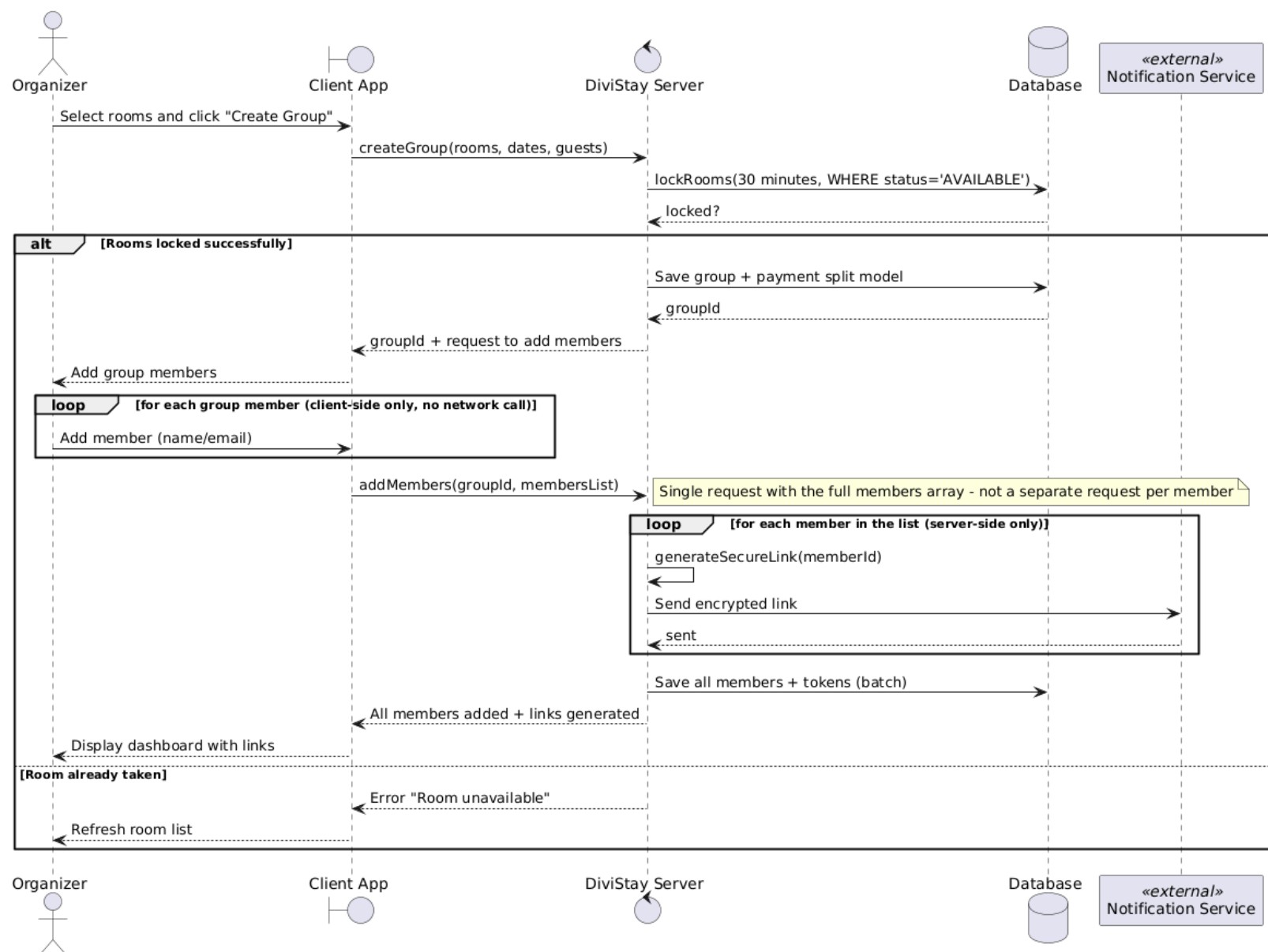
❶ **יצירת קבוצה:** תרשים רצף זה מציג את חילופי ההודעות בין רכיבי המערכת לאורך זמן בעת יצירת הזמנה.

התהליך מתחיל בניסיון של השרת לנעול חדרים במסד הנתונים, ומתפצל לשני תרחישים:

❷ **הצלחה (החדרים ננעלו):** השרת שומר את הקבוצה ומודל הפיצול. בשלב צירוף החברים הוטמע תכנון הנדסי יעיל למניעת עומס רשת: האפליקציה אוספת את פרטי המוזמנים מקומית (לולאת צד לקוח בלבד), ושולחת בקשה מרוכזת אחת לשרת. השרת בתורו מפעיל לולאה פנימית לייצור קישורים מוצפנים, שולח אותם דרך שירות ההתראות החיצוני, ומבצע שמירה מרוכזת של כלל הנתונים במסד הנתונים לבסוף, מוצג לוח הבקרה למארגן.

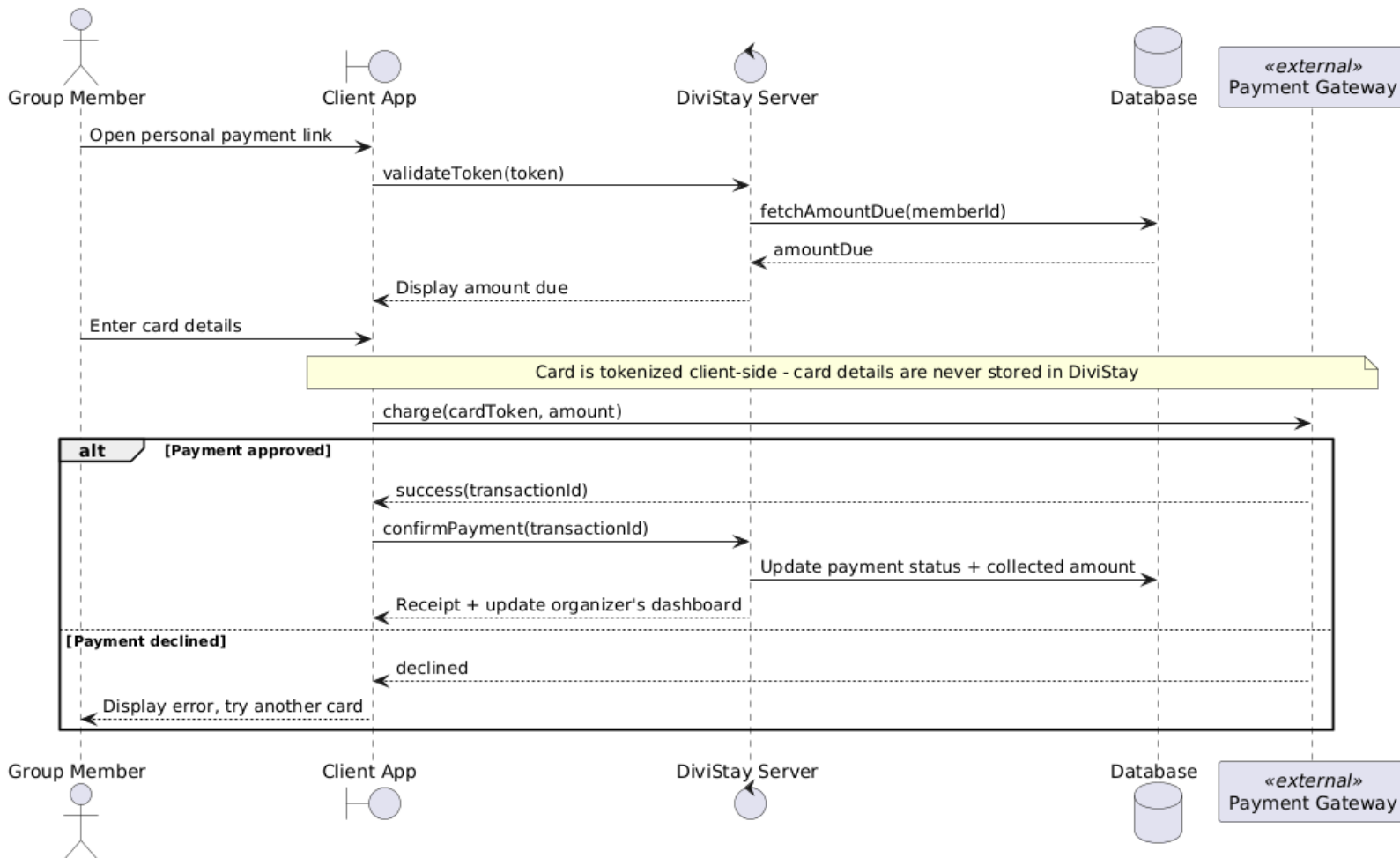
❸ **כישלון (החדר תפוס):** השרת מחזיר שגיאה לאפליקציה, ורשימת החדרים הפנויים מתרעננת למארגן בבחירה מחדש

Group Creation - DiviStay

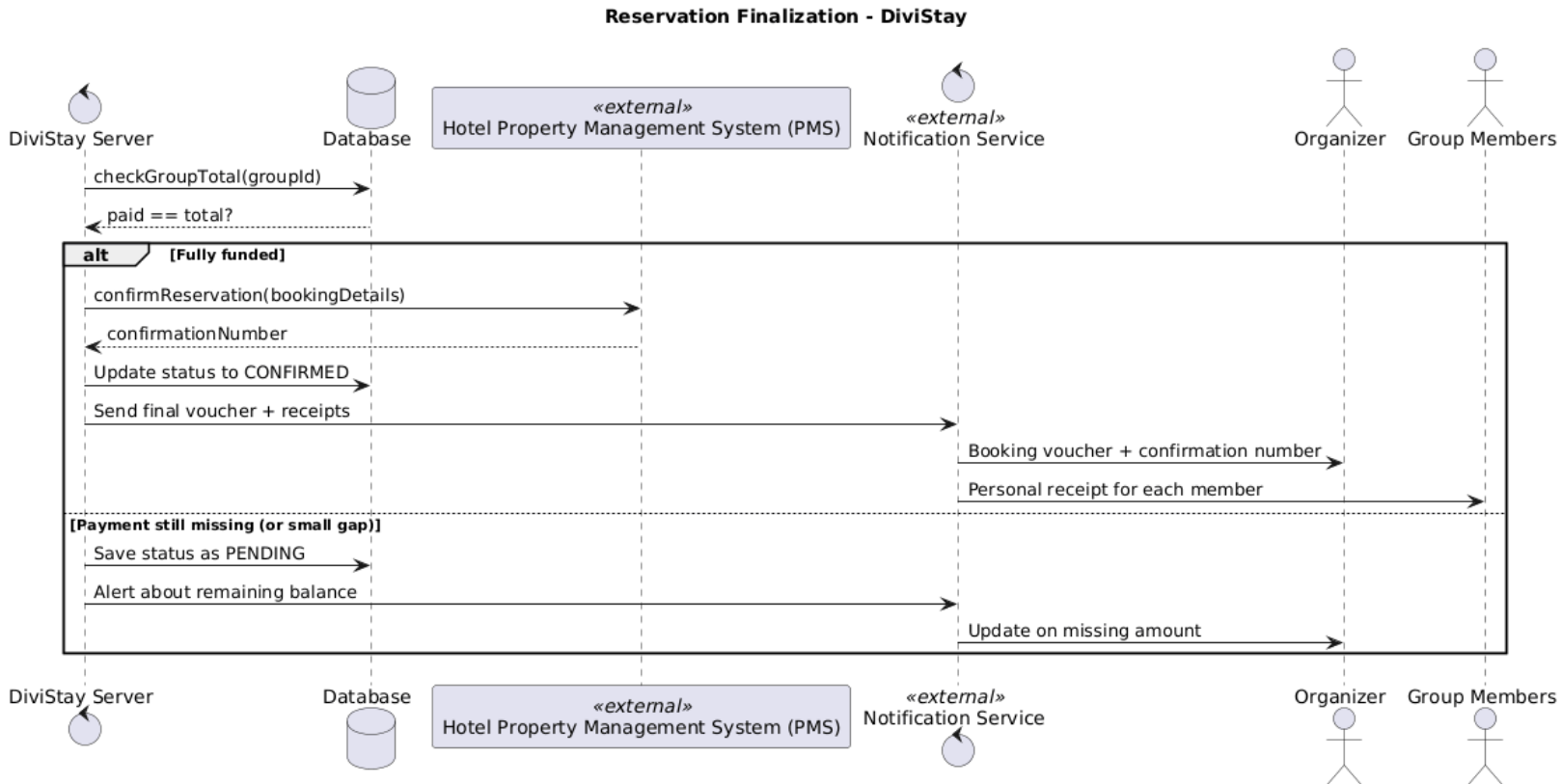


- **תשלום פיצול אישי:** תרשים רצף זה מציג את חילופי ההודעות בין רכיבי המערכת בעת ביצוע תשלום על ידי חבר קבוצה. לאחר אימות הקישור ושליפת הסכום ממסד הנתונים, פרטי האשראי מועברים ישירות לשער תשלומים חיצוני. התהליך מתפצל בהתאם לתשובת שער התשלומים:
- ❖ **התשלום אושר:** השרת מעדכן את סטטוס התשלום במסד הנתונים, מפיק קבלה לחבר הקבוצה ומעדכן את לוח הבקרה של המארגן.
- ❖ **התשלום נדחה:** מוחזרת שגיאה מהשער החיצוני, והאפליקציה מציגה לחבר הודעה המבקשת לנסות כרטיס אשראי אחר.

Split Payment - DiviStay



- **אישור הזמנה סופי:** תרשים רצף זה מציג את תהליך הסנכרון והאישור הסופי מול המלון. התהליך מתחיל בבדיקה של השרת מול מסד הנתונים האם הקבוצה מומנה במלואה, ומתפצל בהתאם לתוצאה:
 - ❖ **הקבוצה שילמה במלואה:** השרת שולח אישור הזמנה למערכת ניהול המלון החיצונית. עם קבלת מספר האישור, הסטטוס מתעדכן ל"מאושר" במסד הנתונים, ושירות ההתראות מפיק שובר אירוח למארגן וקבלות אישיות לכלל החברים.
 - ❖ **קיים חוסר בתשלום:** הסטטוס נשמר כ"ממתין" במסד הנתונים, והשרת מפעיל את שירות ההתראות כדי לשלוח למארגן עדכון על פער התשלום שנותר להשלם.



• Class diagrams

דיאגרמת ה-Class Diagram של DiviStay מוצגת בשלושה חלקים. שלושת החלקים יחד מרכיבים את ה-Domain Model המלא של המערכת (ללא שכבת ה-Controllers/Services, המוצגת בדיאגרמה נפרדת):

❖ חלק 1 - משתמשים, מלון וחדרים:

חלק זה מציג את מבנה המשתמשים והמלון. המודל מיישם עיצוב גמיש מבוסס תפקידים: במקום שימוש בירושה נוקשה, קיימת מחלקת User המייצגת משתמש כללי במערכת. המשתמש יכול לתפקד כמארגן הקבוצה (מיוצג באמצעות קשר ישיר בשם organizer מ- BookingGroup ל- User או כמשתתף (באמצעות מחלקת הקישור GroupMembership)). תכנון הנדסי זה מאפשר לאותו אדם לשמש כמארגן בקבוצה אחת וכחבר בקבוצה אחרת בו זמנית באופן טבעי. במקביל, המחלקה Room שייכת למחלקה Hotel ביחס הכלה חזקה כך שמחיקת המלון מוחקת גם את חדריו. מחלקת RoomFactory מיישמת את תבנית ה- Factory ליצירת אובייקטי חדר.

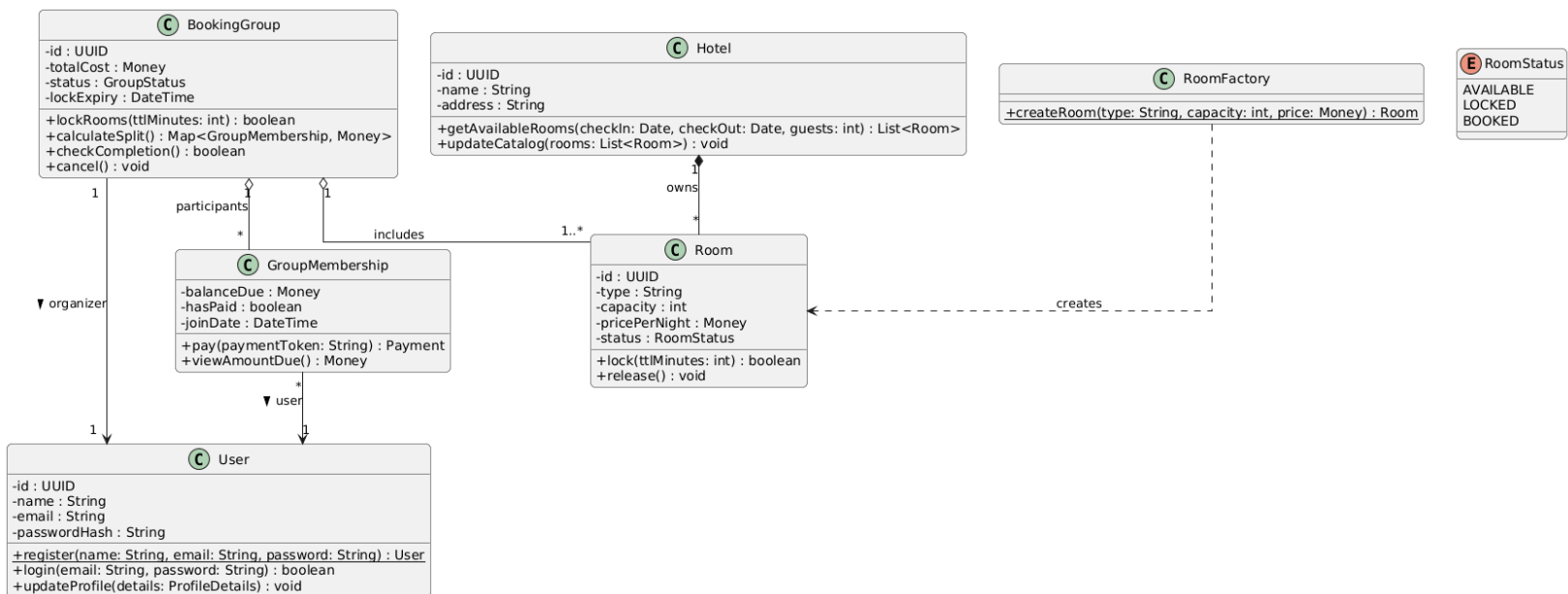
❖ חלק 2 - קבוצת הזמנה ואסטרטגיית פיצול:

חלק זה ממוקד ב- BookingGroup המאגדת משתתפים וחדרים הקיימים באופן עצמאי באמצעות יחס צבירה ולכן הם אינם נמחקים אם הקבוצה מבוטלת. לעומת זאת, אובייקט ה- PaymentSplitStrategy המשווין לקבוצה הוא בבעלותה הבלעדית שכן כל קבוצה מקבלת מופע אסטרטגיה ייעודי הנוצר עבורה על ידי SplitStrategyFactory. שלוש המחלקות המממשות את הממשק (EqualSplitStrategy, RoomBasedSplitStrategy, CustomSplitStrategy) מדגימות את תבנית ה- Strategy למנגנון פיצול תשלום גמיש.

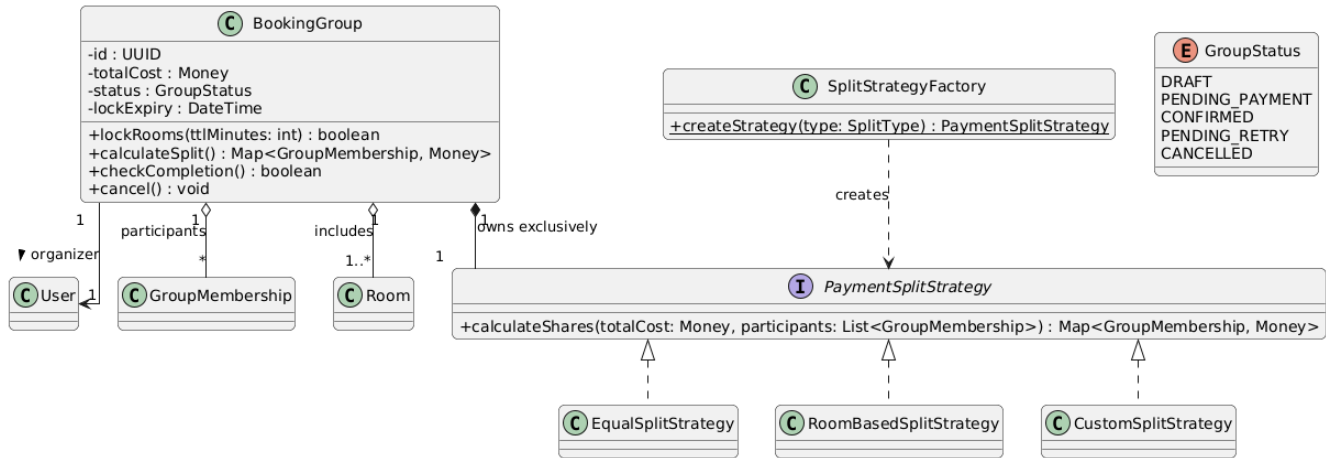
❖ חלק 3 - תשלומים:

חלק זה מתאר את שכבת התשלומים. GroupMember מבצע תשלומים המשוויכים לקבוצה ביחס של הכלה חלשה, המאפשר לרשומות התשלום להמשיך להתקיים למטרות ארכיון היסטורי גם לאחר סיום או ביטול הקבוצה. המחלקה PaymentGatewayAdapter מיישמת את תבנית ה- Adapter, ובכך מבודדת את הלוגיקה הפנימית של המערכת מפרטי ה- API של ספק הסליקה החיצוני, דבר אשר מאפשר החלפת ספק סליקה בעתיד מבלי לשנות את קוד המערכת הקיים.

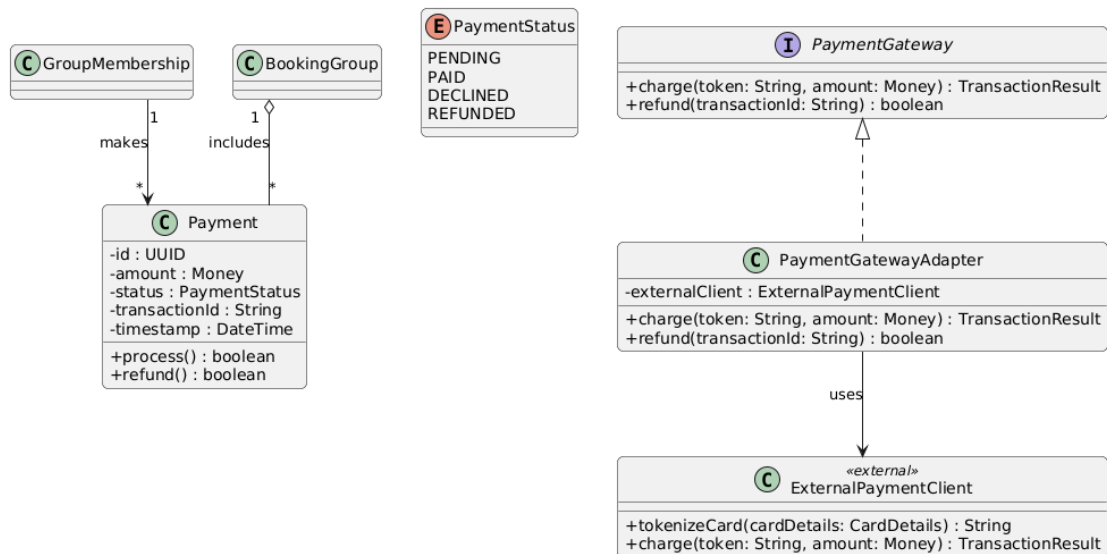
DiviStay — Class Diagram (Domain Model) — Part 1 of 3: Users, Hotel & Rooms (Fixed)



DiviStay — Class Diagram (Domain Model) — Part 2 of 3: Booking Group & Split Strategy



DiviStay — Class Diagram (Domain Model) — Part 3 of 3: Payments



Symbol	Name	Meaning
C	Class	concrete object, has fields (-) and methods (+)
I	Interface	contract only, no fields, implemented by a class
E	Enum	fixed list of named constant values
— (plain line)	Association	one object simply references/uses another
—◇ (hollow diamond, on the "whole" side)	Aggregation	"has-a"; the part can exist without the whole
—◆ (filled diamond, on the "whole" side)	Composition	"owns"; the part is destroyed with the whole
---> (dashed line, open arrowhead)	Dependency	one class calls/creates another, no stored reference
--- > (dashed line, hollow triangle)	Realization	a class implements an interface
1, *, 1..*, 0..1	Multiplicity	how many objects take part on that end of the relation
«external»	Stereotype	marks a class as third-party / outside our own domain code

• Design Patterns

○ תבנית עיצוב Strategy - מודל פיצול תשלומים

בבחירת תבנית ה - Strategy פתרנו את הצורך המרכזי במערכת **DiviStay** לתמוך במספר שיטות חלוקת תשלום שונות בין חברי הקבוצה: חלוקה שווה, חלוקה יחסית לפי מחיר החדר, וחלוקה מותאמת אישית מבלי לקשור את הלוגיקה העסקית של BookingGroup למימוש ספציפי של אחת השיטות.

○ מבנה התבנית:

❖ **ממשק:** התבנית מוגדרת סביב הממשק PaymentSplitStrategy המצהיר על מתודה יחידת (totalCost, participants) calculateShares המחזירה מיפוי בין כל חבר קבוצה לסכום שעליו לשלם.

❖ **מימושים קונקרטיים:** שלוש מחלקות קונקרטיות EqualSplitStrategy, RoomBasedSplitStrategy ו- CustomSplitStrategy ממשות ממשק זה, כאשר כל אחת מהן מיישמת לוגיקת חלוקה שונה.

❖ **הקשר:** מחלקת BookingGroup אינה מכירה כלל את סוג האסטרטגיה הקונקרטית. היא מחזיקה הפניה יחידה, בהכלה חזקה לאובייקט מטיפוס הממשק בלבד, ומשפיעה את חישוב הפיצול בכל קריאה למתודה calculateSplit().

❖ **מפעל:** יצירת מופע האסטרטגיה המתאימה, בהתאם לבחירת המארגן, מתבצעת על ידי SplitStrategyFactory, כך שגם החלטת ה- instantiation אינה מוטלת על ההקשר.

